



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1 по дисциплине «Структуры и алгоритмы обработки данных»

Тема: реализация АД на статическом массиве

Выполнил студент группы ИКБО-
64-23

Астахов С.П.

Принял старший преподаватель
кафедры

Скворцова Л.А

Москва 2024

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ	3
2 РАЗРАБОТКА ПРОГРАММЫ.....	4
2.1 Реализация данных АТД	4
2.1 Реализация алгоритма на языке С++ (1 задача)	7
2.2 Тестирование дополнительной операции	16
2.3 Реализация алгоритма на языке С++ (2 задача)	17
2.4 Тестирования дополнительной операции на vector	25
3 ВЫВОДЫ	26
4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	27

1 ПОСТАНОВКА ЗАДАЧИ И УСЛОВИЕ

Цели работы:

- приобретение умений и навыков определения и реализации абстрактного типа данных задачи в программе;
- приобретение навыков по реализации алгоритмов операций над массивом через аппарат функций языка C++;
- приобретение навыков реализации операций модификации (вставка и удаление элементов) одномерного массива и других операций обработки массива как структуры данных.

Задачи:

1. Реализовать АТД задачи, разработанное в практической работе 1, используя для представления значений множества динамический массив.
2. Выполнить реализацию АТД задачи на динамическом массиве. Для управления динамической памятью использовать функции файла заголовка `stdlib.h`: `malloc`, `free`, `realloc`. (1 задача)
3. Реализовать АТД задачи, разработанное в задании 1 практической работы, используя для представления структуры данных современный динамический массив языка C++ — вектор (`vector`) – контейнер STL C++. (2 задача)
4. Индивидуальный вариант №6: сформировать новый массив из чисел исходного, у которых в записи числа нет цифры 3 (для двух задач).

2 РАЗРАБОТКА ПРОГРАММЫ

2.1 Реализация данных АТД

Для разработки решения задачи (нашего АТД) составим его описание, в котором будут определены основные поля, функции для выполнения алгоритма прошлой практической работы и работы индивидуального варианта 3 практической (см. лист 3)

АТД *staticTypeData*

Данные:

A – список значений элементов множества;

length_arr – кол-во элементов множества;

Операции:

append_capacity(a) { // Для 1 части практики

Операция: повышает размер элементов множества;

Предусловие: staticTypeData a

Постусловие: возвращение увеличенного множества

}

setFill(a, n, choise) {

Операция: заполняет множество на n элементов, введенных с клавиатуры или случайным образом;

Предусловие: множество элементов A, число целого типа n и choise

Постусловие: заполняет множество a.A элементами размера n

}

Print(set) {

Операция: выводит множество в консоль;

Предусловие: множество элементов A

Постусловие: поэлементно выводит у множества значения

}

isFirstOrderliness (num) {

Операция: проверяет является ли у числа все разряды(цифры) упорядочены;

Предусловие: число целого типа num

Постусловие: true – если все разряды числа упорядочены по возрастанию,
иначе — false

}

numPosition(a) {

Операция: ищет элемент, соответствующий условию задачи;

Предусловие: множество элементов A

Постусловие: номер элемента если он найден, в ином случае -1

}

push_num (num, pos, &a) {

Операция: делает вставку элемента на позицию следующего после числа;

Предусловие: множество элементов A, число целого типа num и pos

Постусловие: добавляет элемент после элемента, удовлетворяющего
операции isFirstOrderliness

}

deleteNum(pos, &a) {

Операция: удаляет перед элементом, если нынешний элемент удовлетворяет
условию задачи;

Предусловие: множество элементов A, число целого типа pos

Постусловие: удаляет элемент перед элементом, который удовлетворяет
операции isFirstOrderliness

```

}

new_array_not3(staticTypeData &a) {
    Операция: удаление всех чисел, у которых есть цифра “3”
    Предусловие: множество элементов A
    Постусловие: создает новое множество без чисел имеющие цифру “3”
}

append_push(staticTypeData& a, int num) {
    Операция: добавления значения в массив в конец
    Предусловие: множество элементов A, число целого типа num
    Постусловие: добавляет элемент в конец множества
}

void append_in_pos(staticTypeData& a, int num, int add_pos){
    Операция: добавления значения в массив в выбранной позиции
    Предусловие: множество элементов A, число целого типа num и add_pos
    Постусловие: добавляет элемент в множество
}

void remove_in_pos(staticTypeData& a, int mov_pos){
    Операция: удаление значения массива в выбранной позиции
    Предусловие: множество элементов A, число целого типа mov_pos
    Постусловие: удаляет элемент в множестве
}

clearData(staticTypeData &a) { // Для 1 части практики
    Операция: освобождает память динамического массива
    Предусловие: множество элементов A
    Постусловие: удаляет множество (подчищает память)
}

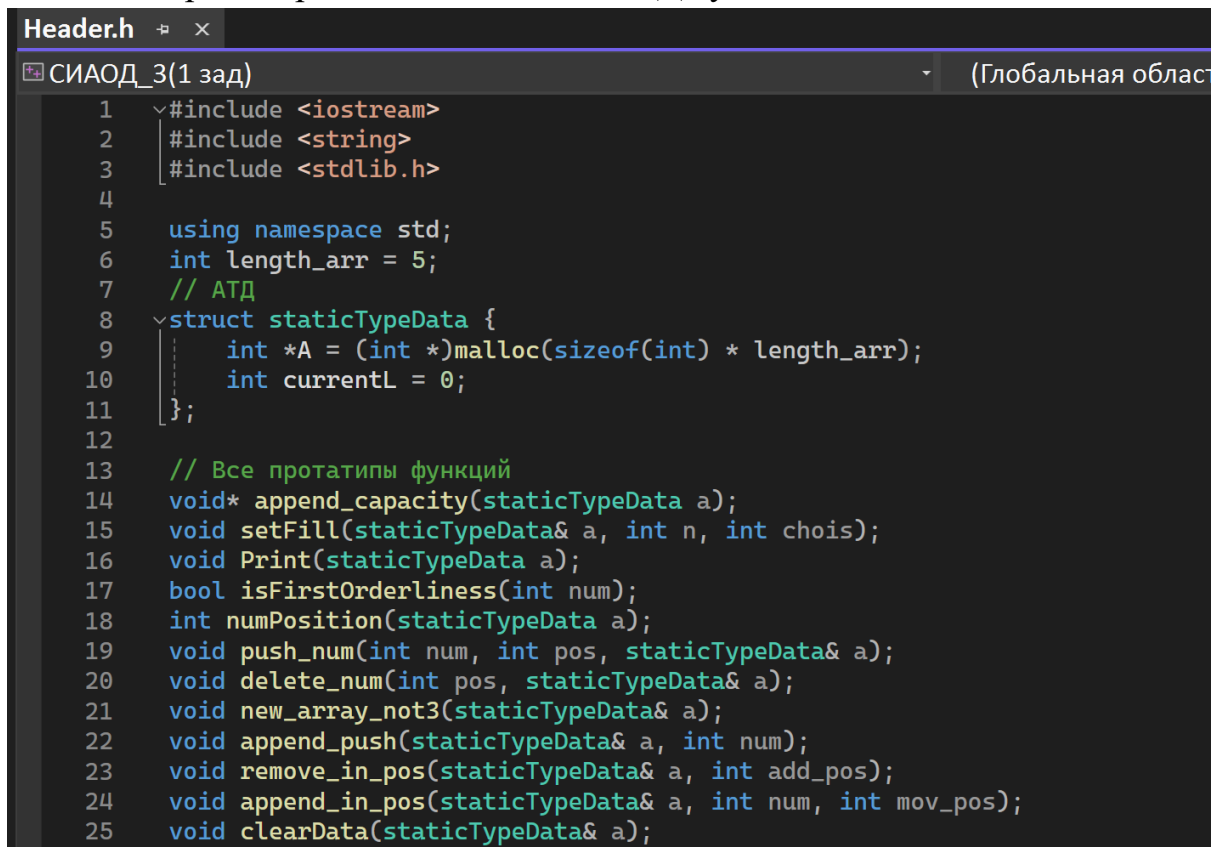
```

```
}
```

2.1 Реализация алгоритма на языке C++ (1 задача)

Как и в прошлой работе для реализации АДД используем структуру данных `struct` стандартной библиотеки языка C++, внутри ее определим описанные выше поля для хранения динамического массива и его длины. В отдельном (заголовочном) файле реализуем методы для работы со динамическим массивом структуры (вставка, удаление, поиск), в главном файле реализуем системы ввод-вывод с помощью оператора `switch-case`, предварительно подключив заголовочный файл через `#include`. Фактически переделаем файлы предыдущей работы под динамический массив, для скорости написания программы.

Таким образом, реализация данных АДД будет выглядеть так:



```
Header.h  + x
СИАОД_3(1 зад)  (Глобальная область)

1  #include <iostream>
2  #include <string>
3  #include <stdlib.h>
4
5  using namespace std;
6  int length_arr = 5;
7  // АДД
8  struct staticTypeData {
9      int *A = (int *)malloc(sizeof(int) * length_arr);
10     int currentL = 0;
11 };
12
13 // Все прототипы функций
14 void* append_capacity(staticTypeData a);
15 void setFill(staticTypeData& a, int n, int chois);
16 void Print(staticTypeData a);
17 bool isFirstOrderliness(int num);
18 int numPosition(staticTypeData a);
19 void push_num(int num, int pos, staticTypeData& a);
20 void delete_num(int pos, staticTypeData& a);
21 void new_array_not3(staticTypeData& a);
22 void append_push(staticTypeData& a, int num);
23 void remove_in_pos(staticTypeData& a, int add_pos);
24 void append_in_pos(staticTypeData& a, int num, int mov_pos);
25 void clearData(staticTypeData& a);
```

Рис. 1 – struct для АДД

Алгоритм индивидуального варианта №6:

```
void new_array_not3(staticTypeData &a) { // Задание 3 ПР
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }

    for (int i = a.currentL - 1; i >= 0; i--) {
        if (to_string(a.A[i]).find('3') != string::npos) {
            a.A[i] = 0;
            if (i != a.currentL){
                for (int j = i; j < a.currentL; j++) {
                    a.A[j] = a.A[j + 1];
                }
            }
            a.currentL -= 1;
        }
    }
}
```

Заметим, что перед тем, как использовать размещение динамического блока памяти malloc и других функций для получения динамического массива, нужно подключить библиотеку <stdlib.h>.

Дальнейшая реализация заголовочного файла будет такой:

```

// Реализция функций
void *append_capacity(staticTypeData a) {
    (length_arr) *= 2;
    int* ar = (int*)realloc(a.A, sizeof(int) * length_arr);
    if (ar == NULL) {
        return a.A;
    }
    a.A = ar; // realloc - автоматически подчищает память

    return a.A;
}

void setFill(staticTypeData& a, int n, int choose) {

    if ( (a.currentL + n) > length_arr) {
        a.A = (int *)append_capacity(a);
    }
    int num;
    if (choose == 1) {
        cout << "Ввод чисел:" << endl;
        for (int i = 0; i < n; i++)
        {
            cout << i + 1 << ") "; cin >> num;
            a.A[a.currentL] = num;
            a.currentL += 1;
        }
    }
    else {
        srand(time(0));
        for (int i = 0; i < n; i++)
        {
            num = rand() % 201 + 100;
            a.A[a.currentL] = num;
            a.currentL += 1;
        }
    }
}

void Print(staticTypeData a) {
    if (a.currentL == 0) {
        cout << "нет элементов в массиве" << endl;
        return;
    }
}

```

Рис.2 – Часть 1 заголовочного файла

```

void Print(staticTypeData a) {
    if (a.currentL == 0) {
        cout << "нет элементов в массиве" << endl;
        return;
    }
    cout << "Элементы: " << endl;

    for (int i = 0; i < a.currentL; i++)
    {
        cout << a.A[i] << endl;
    }
    cout << "Длина: " << a.currentL << endl;;
}

bool isFirstOrderliness(int num) { // 1 Задание(2 ПР)
    // 1234 -> 1234%10 = 4(m), 1234/10%10=3(num); 4<=3
    num = abs(num);
    int m = num % 10;
    while (num /= 10) {
        if (m <= num % 10) {
            return false;
        }
        m = num % 10;
    }
    return true;
}

int numPosition(staticTypeData a) {

    for (int i = 0; i < a.currentL; i++) {
        if (isFirstOrderliness(a.A[i])) {
            return i;
        };
    }
    return -1;
}

void push_num(int num, int pos, staticTypeData &a) { // 2 Задание(2 ПР)

    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }
}

```

Рис.3 – Часть 2 заголовочного файла

```

void push_num(int num, int pos, staticTypeData &a) { // 2 Задание(2 ПР)

    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }
    if (length_arr == a.currentL) {
        append_capacity(a);
    }
    // Добавления числа после поз.числа
    for (int i = a.currentL; i >= pos; i--) {
        a.A[i + 1] = a.A[i];
    }

    a.A[pos + 1] = num;
    a.currentL += 1;
}

void delete_num(int pos, staticTypeData& a) { // 3 Задание(2 ПР)
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }
    else if (pos == 0) {
        cout << "Нельзя удалить число, позиционное число первое в массиве";
        return;
    }
    // Удаления числа перед поз.числом
    int i = 0;
    while (i < a.currentL) {
        if (isFirstOrderliness(a.A[i + 1])) {
            for (int j = i; j < a.currentL; j++) {
                a.A[j] = a.A[j + 1];
            }
            a.currentL -= 1;
            i += 1;
        }
        else {
            i += 1;
        }
    }
}

```

Рис.4 – Часть 3 заголовочного файла

```

void new_array_not3(staticTypeData &a) { // Задание 3 ПР
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }

    for (int i = a.currentL - 1; i >= 0; i--) {
        if (to_string(a.A[i]).find('3') != string::npos) {
            a.A[i] = 0;
            if (i != a.currentL){
                for (int j = i; j < a.currentL; j++) {
                    a.A[j] = a.A[j + 1];
                }
            }
            a.currentL -= 1;
        }
    }
}

void append_push(staticTypeData& a, int num) {
    // Операция добавления элемента с помощью библиотеки stdlib.h(3 ПР)
    if (length_arr == a.currentL) {
        a.A = (int*)append_capacity(a);
    }

    if (a.currentL == 0) {
        a.A[a.currentL] = num;
        a.currentL += 1;
        return;
    }
    a.A[a.currentL + 1] = num;
    a.currentL += 1;
}

void append_in_pos(staticTypeData& a, int num, int add_pos) {
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }
    if (length_arr == a.currentL) {
        a.A = (int*)append_capacity(a);
    }
    //
    for (int i = a.currentL; i >= add_pos; i--) {
        a.A[i + 1] = a.A[i];
    }
    a.A[add_pos] = num;
    a.currentL += 1;
}

```

Рис.5 – Часть 4 заголовочного файла

```

void append_in_pos(staticTypeData& a, int num, int add_pos) {
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }
    if (length_arr == a.currentL) {
        a.A = (int*)append_capacity(a);
    }
    //
    for (int i = a.currentL; i >= add_pos; i--) {
        a.A[i + 1] = a.A[i];
    }
    a.A[add_pos + 1] = num;
    a.currentL += 1;

    //
}

void remove_in_pos(staticTypeData& a, int mov_pos) {
    if (a.currentL == 0) {
        cout << "Пустое множество";
        return;
    }

    int i = 0;
    while (i < a.currentL) {
        if (i == mov_pos) {
            for (int j = i; j < a.currentL; j++) {
                a.A[j] = a.A[j + 1];
            }
            a.currentL -= 1;
            i += 1;
        }
        else {
            i += 1;
        }
    }
}

void clearData(staticTypeData &a) { // Отчищение памяти
    free(a.A);
}

```

Рис.6 – Часть 5 заголовочного файла

В главном файле реализуем интерфейс для ввода и вывода текста в консоль:

```
#include "Header.h"
int main() {
    setlocale(0, "ru");
    staticTypeData set;
    int choise = -1, size = 0, ch = 0;
    while (choise != 0) {
        int pos = numPosition(set);
        cout << "\nВведите функцию:" << endl;
        cout << "1)Заполнить множество числами " << endl;
        cout << "2)Вывести множество" << endl;
        cout << "3)Найти позицию числа" << endl;
        cout << "4)Вставить число после позиционного числа" << endl;
        cout << "5)Вставить число в конце" << endl;
        cout << "6)Удалить число после позиционного числа" << endl;
        cout << "7)Сформировать массив у которого в записи чисел нет цифры 3." << endl;
        cout << "8)Вставить число в выбранную ячейку" << endl;
        cout << "9)Удалить число в выбранной ячейке" << endl;
        cout << "0)Выход из программы" << endl;
        cout << "Ввод: "; cin >> choise;

        if (choise != 0) {
            switch (choise) {
                case 1:
                    cout << "Введите размер массива: "; cin >> size;
                    int choise_answer;
                    cout << "1)Элементы с клавиатуры\n2)Рандомные элементы\n"; cin >> choise_answer;
                    setFill(set, size, choise_answer);
                    break;
                case 2:
                    Print(set);
                    break;
                case 3:
                    if (pos != (-1)) {
                        cout << "Число: " << set.A[pos] << "\nимеет позицию числа: " << pos;
                    }
                    else {
                        cout << "Нет такого числа!";
                    }
                    break;
                case 4:
                    int num;
                    if (pos == -1) {
```

Рис.7 – Часть 1 main

```

        break;
    case 4:
        int num;
        if (pos == -1) {
            cout << "Нет поз. числа" << endl;
            break;
        }
        cout << "Введите число для добавления: "; cin >> num;
        push_num(num, pos, set);
        break;
    case 5:
        cout << "Введите число для добавления: "; cin >> num;
        append_push(set, num);
        break;
    case 6:
        if (pos == -1) {
            cout << "Нет поз. числа" << endl;
            break;
        }
        delete_num(pos, set);
        break;
    case 7:
        new_array_not3(set);
        break;
    case 8:
        cout << "Введите число для добавления: "; cin >> num;
        cout << "Выбор ячейки массива: "; cin >> ch;
        append_in_pos(set, num, ch);
        break;
    case 9:
        cout << "Выбор ячейки массива: "; cin >> ch;
        remove_in_pos(set, ch);
        break;
    default:
        cout << "Ошибка...";
        break;
    }
}

cout << "Подчищение данных..." << endl;
clearData(set);
cout << "Выход из программы..." << endl << endl;
return 0;
}

```

Рис.8 – Часть 2 main

2.2 Тестирование дополнительной операции

Для того, чтобы убедиться в корректности работы алгоритма необходимо разработать тесты (задать определенные значения, найти ожидаемый результат) и затем сравнить результаты ручного тестирования и выполнения программы на компьютере. Для заноса результатов индивидуального задания, создадим одну таблицу и сделаем фотки с visual studio (среда разработки на c++). Так как мой индивидуальный вариант 6, мне надо было удалить все числа, имеющие цифру “3”, тем самым, переделав старый массив.

Номер теста	Входные данные	Эталон результата
1	n = 5, A {100, 200, 339, 231, 211}	n = 3, A {100, 200, 211}
2	n = 6, A {100, 200, 500, 600, 500, 334}	n = 5, A {100, 200, 500, 600, 500,}
3	n = 4, A {300,330, 339, 324}	n = 0, A {}

```
Элементы:
100
200
211
Длина: 3
```

Рис.9 – Результат 1 по функции new_array_not3

```

Элементы:
100
200
500
600
500
Длина: 5

```

Рис.10 – Результат 2 по функции new_array_not3

НЕТ ЭЛЕМЕНТОВ В МАССИВЕ

Рис.11 – Результат 3 по функции new_array_not3

2.3 Реализация алгоритма на языке C++ (2 задача)

Вторая задача подразумевает работа с динамическим массивом при помощи препроцессора vector. В данной задаче будет легче написать код из-за наличие изначальных методов нахождения, удаление, добавления элементов.

Итак, можно сравнить функции для первого задания и второго, где функции выполняют какие-то прописанные методы vector

Операция	Коды функций задания 1	Коды функций задания 2
Вставить элемент	<pre> if (a.currentL == 0) { cout << "Пустое множество"; return; } if (length_arr == a.currentL) { a.A (int*)append_capacity(a); } // for (int i = a.currentL; i >= </pre>	<pre> if (a.A.size() == 0) { cout << "Пустое множе- ство"; return; } // Добавления числа после поз.числа a.A.insert(a.A.begin() + add_pos, num); </pre>

	<pre> add_pos; i--) { a.A[i + 1] = a.A[i]; } a.A[add_pos + 1] = num; a.currentL += 1; </pre>	
Удалить элемент	<pre> if (a.currentL == 0) { cout<<"Пустое множество"; return; } Int i = 0; while (i < a.currentL) { if (i == mov_pos) { for (int j = i; j < a.currentL; j++) { a.A[j] = a.A[j + 1]; } a.currentL -= 1; i += 1; } else { i += 1; } } </pre>	<pre> if (a.A.size() == 0) { cout << "Пустое множество"; return; } else if (mov_pos == 0) { cout << "Нельзя удалить число, позиционное число первое в массиве"; return; } // Удаления числа a.A.erase(a.A.begin() + mov_pos); </pre>
Формирование нового множества	<pre> if (length_arr == a.currentL) { a.A = (int*)append_capacity(a); } if (a.currentL == 0) { a.A[a.currentL] = num; a.currentL += 1; return; } a.A[a.currentL + 1] = num; a.currentL += 1; </pre>	<pre> if (a.A.size() == 0) { cout << "Пустое множе- ство"; return; } vector <int>::iterator Iter; int i = 0; while(i < a.A.size()){ if (to_string(a.A[i]).find('3') != string::npos) { a.A.erase(a.A.begin() + i); } else { i += 1; } } </pre>

Код реализации на vector

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;
// АД
struct staticTypeData {
    vector<int> A;
};
// Все протатипы функций
void* append_capacity(staticTypeData a);
void setFill(staticTypeData& a, int n, int chois);
void Print(staticTypeData a);
bool isFirstOrderliness(int num);
int numPosition(staticTypeData a);
void push_num(int num, int pos, staticTypeData& a);
void delete_num(int pos, staticTypeData& a);
void new_array_not3(staticTypeData& a);
void append_push(staticTypeData& a, int num);
void remove_in_pos(staticTypeData& a, int add_pos);
void append_in_pos(staticTypeData& a, int num, int mov_pos);

// Реализция функций
void setFill(staticTypeData& a, int n, int choise) {
    int num;
    cout << "Ввод чисел:" << endl;
    if (choise == 1) {
        for (int i = 0; i < n; i++)
        {
            cout << i + 1 << ") "; cin >> num;
            a.A.push_back(num);
        }
    }
    else {
        srand(time(0));
        for (int i = 0; i < n; i++)
        {
            num = rand() % 401 + 100;
            a.A.push_back(num);
        }
    }
}

void Print(staticTypeData a) {
```

Рис.12 – Часть 1 заголовочного файла

```

void Print(staticTypeData a) {
    if (a.A.size() == 0) {
        cout << "нет элементов в массиве" << endl;
        return;
    }

    cout << "Элементы: " << endl;

    for (int i = 0; i < a.A.size(); i++)
    {
        cout << a.A[i] << endl;
    }
    cout << "Длина: " << a.A.size() << endl;;
}

bool isFirstOrderliness(int num) { // 1 Задание
    // 1234 -> 1234%10 = 4(m), 1234/10%10=3(num); 4<=3
    num = abs(num);
    int m = num % 10;
    while (num /= 10) {
        if (m <= num % 10) {
            return false;
        }
        m = num % 10;
    }
    return true;
}

int numPosition(staticTypeData a) {

    for (int i = 0; i < a.A.size(); i++) {
        if (isFirstOrderliness(a.A[i])) {
            return i;
        };
    }
    return -1;
}

void push_num(int num, int pos, staticTypeData& a) { // 2 Задание

    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }

    // Добавления числа после поз.числа
    a.A.insert(a.A.begin() + pos + 1, num);
}

```

Рис.13 – Часть 2 заголовочного файла

```

void push_num(int num, int pos, staticTypeData& a) { // 2 Задание

    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    // Добавления числа после поз.числа
    a.A.insert(a.A.begin() + pos + 1, num);
}

void delete_num(int pos, staticTypeData& a) { // 3 Задание
    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    else if (pos == 0) {
        cout << "Нельзя удалить число, позиционное число первое в массиве";
        return;
    }
    // Удаления числа перед поз.числом
    a.A.erase(a.A.begin() + pos - 1);
}

void new_array_not3(staticTypeData& a) {
    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    vector<int>::iterator Iter;
    int i = 0;
    while(i < a.A.size()){
        if (to_string(a.A[i]).find('3') != string::npos) {
            a.A.erase(a.A.begin() + i );
        }
        else {
            i += 1;
        }
    }
}

void append_in_pos(staticTypeData& a, int num, int add_pos) {
    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    // 5
}

```

Рис.14 – Часть 3 заголовочного файла

```

void append_in_pos(staticTypeData& a, int num, int add_pos) {
    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    // Добавления числа после поз.числа
    a.A.insert(a.A.begin() + add_pos, num);
}

void remove_in_pos(staticTypeData & a, int mov_pos) {
    if (a.A.size() == 0) {
        cout << "Пустое множество";
        return;
    }
    else if (mov_pos == 0) {
        cout << "Нельзя удалить число, позиционное число первое в массиве";
        return;
    }
    // Удаления числа
    a.A.erase(a.A.begin() + mov_pos);
}

void append_push(staticTypeData& a, int num) {
    a.A.push_back(num);
}

```

Рис.15 – Часть 4 заголовочного файла

```

#include "Header.h"
int main() {
    setlocale(0, "ru");
    static TypeData set;
    int choose = -1, size = 0, ch = 0;
    while (choose != 0) {
        int pos = numPosition(set);
        cout << "\nВведите функцию:" << endl;
        cout << "1)Заполнить множество числами " << endl;
        cout << "2)Вывести множество" << endl;
        cout << "3)Найти позицию числа" << endl;
        cout << "4)Вставить число после позиционного числа" << endl;
        cout << "5)Вставить число в конце" << endl;
        cout << "6)Удалить число после позиционного числа" << endl;
        cout << "7)Сформировать массив у которого в записи чисел нет цифры 3." << endl;
        cout << "8)Вставить число в выбранную ячейку" << endl;
        cout << "9)Удалить число в выбранной ячейке" << endl;
        cout << "0)Выход из программы" << endl;
        cout << "Ввод: "; cin >> choose;

        if (choose != 0) {
            switch (choose) {
                case 1:
                    cout << "Введите размер массива: "; cin >> size;
                    int choose_answer;
                    cout << "1)Элементы с клавиатуры\n2)Рандомные элементы\n"; cin >> choose_answer;
                    setFill(set, size, choose_answer);
                    break;
                case 2:
                    Print(set);
                    break;
                case 3:
                    if (pos != (-1)) {
                        cout << "Число: " << set.A[pos] << "\nhимеет позицию числа: " << pos;
                    }
                    else {
                        cout << "Нет такого числа!";
                    }
                    break;
                case 4:
                    int num;
                    if (pos == -1) {
                        cout << "Нет поз. числа" << endl;
                        break;
                    }
                    cout << "Введите число для добавления: "; cin >> num;
                    push_num(num, pos, set);
            }
        }
    }
}

```

Рис.16 – Часть 1 main

```

        int num;
        if (pos == -1) {
            cout << "Нет поз. числа" << endl;
            break;
        }
        cout << "Введите число для добавления: "; cin >> num;
        push_num(num, pos, set);
        break;
    case 5:
        cout << "Введите число для добавления: "; cin >> num;
        append_push(set, num);
        break;
    case 6:
        if (pos == -1) {
            cout << "Нет поз. числа" << endl;
            break;
        }
        delete_num(pos, set);
        break;

    case 7:
        new_array_not3(set);
        break;

    case 8:
        cout << "Введите число для добавления: "; cin >> num;
        cout << "Выбор ячейки массива: "; cin >> ch;
        append_in_pos(set, num, ch);
        break;

    case 9:
        cout << "Выбор ячейки массива: "; cin >> ch;
        remove_in_pos(set, ch);
        break;

    default:
        cout << "Ошибка...";
        break;
    }
}

cout << "Выход из программы..." << endl << endl;
return 0;
}

```

Рис.17 – Часть 2 main

2.4 Тестирования дополнительной операции на vector

Сделаем такую же таблицу, как и для задания 1.

Номер теста	Входные данные	Эталон результата
1	n = 5, A {100, 200, 339, 231, 211}	n = 3, A {100, 200, 211}
2	n = 6, A {100, 200, 500, 600, 500, 334}	n = 5, A {100, 200, 500, 600, 500}
3	n = 4, A {300, 330, 339, 324}	n = 0, A {}

```
100
200
211
Длина: 3
```

Рис.16 – Результат 1 по функции new_array_not3

```
100
200
500
600
500
Длина: 5
```

Рис.17 – Результат 1 по функции new_array_not3

НЕТ ЭЛЕМЕНТОВ В МАССИВЕ

Рис.18 – Результат 1 по функции new_array_not3

Все результаты совпали, что говорит о правильности программы в двух задачах.

3 ВЫВОДЫ

В ходе данной практической работы мы реализовали свой абстрактный тип данных в соответствие со своим индивидуальным вариантом, реализовали метод для решения индивидуального варианта, научились лучше понимать динамические массивы и приобрели навык по реализации алгоритмов над массивами.

4 СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1) Скворцова Л.А, Гусев К.В, Трушин С.М, Филатов А.С: Учебно-методическое пособие СиАОД часть 1 / — МИРЭА — Российский технологический университет, 2021. — 146 с

2) Лекционный материал Скворцовой Л.А (старший преподаватель кафедры МОСИТ)